

Table des matières

Synthèse — « La Brûlerie », un torréfacteur, de bout en bout	1
Étape 0 — Le projet et son architecture (<i>démarche du Cours 00</i>)	1
Étape 1 — Le brut figé, chargé par dbt seed (<i>démarche du Cours 01</i>).....	6
Étape 2 — L'étoile, sur papier (<i>démarche du Cours 02</i>)	9
Étape 3 — STAGING en dbt : le brut, rendu lisible (<i>démarche du Cours 03</i>)	12
Étape 4 — MARTS en dbt : l'étoile, assemblée (<i>démarche du Cours 04</i>)	15
Étape 5 — La confiance : tests, historique, documentation (<i>démarche du Cours 04</i>)	17
Étape 6 — Le faire tourner (<i>démarche du Cours 05</i>)	19
Récapitulatif — tout l'arc, en un tableau.....	19
Annexe — l'arborescence du projet et les commandes	20

Synthèse — « La Brûlerie », un torréfacteur, de bout en bout

Projet synthèse (facile) — Formation Analytics Engineering (démarches des Cours 00 → 05)

Support d'avant la refonte. La formation a été refondue le 2026-09-20, de Snowflake vers DuckDB et dbt-duckdb, sur des univers redessinés. Cette synthèse porte encore l'ancienne pile, et son projet rejouable a été retiré du dossier en attendant sa réécriture. Les cours 01 à 06, eux, sont à jour. Voir README.md.

Étape 0 — Le projet et son architecture (*démarche du Cours 00*)

0.1 L'entreprise, et comment lire cette synthèse

La Brûlerie est un petit torréfacteur de café. Elle achète du café vert, le torréfie et le vend à des cafés-clients (des commerces, pas le grand public), répartis dans quelques régions. Tout est minuscule à dessein : trois clients, trois produits, cinq lignes de vente. Un modèle qu'on suit du doigt, du premier octet brut jusqu'au chiffre du rapport.

Un seul processus métier : la vente. C'est l'activité dont la patronne veut suivre les chiffres, combien, à qui, où, avec quelle marge. Un seul processus, donc une seule étoile.

Ce document refait tout le trajet (du brut figé jusqu'à l'étoile testée, historisée et documentée) sur un cas si petit qu'on recompte chaque chiffre de tête. Il ne réexplique pas la théorie : il exécute les démarches des Cours 00 à 05, en pointant au passage la section de cours qui les fonde. Gardez l'aide-mémoire ouvert à côté.

Le dénuement est le sujet. Sur un cas qu'on embrasse d'un regard, la mécanique se voit. La Brûlerie reste donc sur le socle RAW → STAGING → MARTS : ni INTERMEDIATE ni PRESENTATION, ces couches n'apparaissent que là où un besoin les appelle.

Sa marque de fabrique est le chargement par dbt seed. Le brut est figé dans de petits CSV versionnés, rangés dans seeds/, que dbt charge lui-même. C'est la façon la plus simple de faire entrer une donnée dans un entrepôt, et la fonction seed, qu'on n'avait pas encore montrée.

En trois lignes : six petites tables qui tiennent à l'écran ; dbt seed charge le brut figé vers RAW, puis STAGING nettoie et MARTS modélise ; à l'arrivée, une étoile fct_ventes entourée de dim_client, dim_produit et dim_date, testée, historisée et documentée.

0.2 La source : le logiciel de caisse

La Brûlerie enregistre ses ventes dans un logiciel de caisse (l'équivalent, à son échelle, de l'ERP SAP de Boréal). Comme tout logiciel réel, la caisse ne stocke pas des noms bien propres : elle a ses abréviations techniques (cli_id, reg_id...), ses codes (seg), et (grand classique) des dates rangées en texte. Sa base : six petites tables, qu'on lit d'un coup d'œil.

region — les régions commerciales

reg_id	reg_nom
1	Amérique
2	Europe

pays — chaque pays est dans une région (reg_id pointe vers region)

pays_id	pays_nom	reg_id
1	Canada	1
2	France	2

client — seg est un code (P = particulier, PRO = professionnel)

cli_id	cli_nom	pays_id	seg
42	Alice	1	P
57	Bruno	2	PRO
88	Carla	2	P

produit

prod_id	prod_nom	cat
5	Café	Boisson
8	Thé	Boisson
9	Tasse	Accessoire

commande — l'en-tête d'une vente ; cmd_date est stockée en texte, pas en date

cmd_id	cmd_date	cli_id
100	2024-01-15	42
101	2024-01-15	57
102	2024-03-02	88

ligne — le détail d'une commande : un produit par ligne (quantité, prix, remise)

cmd_id	lig_no	prod_id	qte	prix	rem
100	1	5	10	3.00	0.10
100	2	9	2	5.00	0.00
101	1	8	5	4.00	0.00
102	1	5	8	3.00	0.05
102	2	8	3	4.00	0.00

Repérez les liens (les colonnes en ..._id) : ce sont eux qui relient les tables. Ce brut est notre point de départ ; on ne le retouchera jamais.

0.3 L'architecture appliquée à La Brûlerie

Le socle de la théorie compte quatre couches, RAW → STAGING → MARTS → BI (la taxonomie complète en ajoute deux, INTERMEDIATE et PRESENTATION, dont La Brûlerie n'a pas besoin). Chaque table de la caisse va parcourir ce socle de bout en bout :

caisse (CSV figés) → RAW (seeds, copiés tels quels) → STAGING (une vue stg_) → MARTS (l'étoile) → Power BI

Couche	Ce qu'elle devient pour La Brûlerie	Bâtie à l'étape
RAW — le brut tel quel	les 6 tables de la caisse, chargées sans retouche par dbt seed (cli_id, seg, dates en texte)	1
STAGING — nettoyé 1 pour 1	6 vues stg_*, une par table : renommer, typer, décoder	3
MARTS — l'étoile métier	un fait fct_ventes entouré de dim_client, dim_produit, dim_date	2 (conçue), 4 (bâtie)
BI — la vue métier	ventes par région, par produit, par mois	(hors périmètre ici)

Pourquoi seule MARTS se conçoit. C'est la seule couche qui demande des décisions (quel grain ? quels faits, quelles dimensions ?). Ce plan se fait « sur papier » à l'étape 2, avant d'être bâti à l'étape 4. Les autres couches n'ont rien à concevoir : RAW copie le brut, STAGING renomme 1 pour 1, elles se bâtissent directement.

Où chaque table aboutit dans l'étoile :

Table de la caisse	Vue STAGING	Nourrit, dans l'étoile
client	stg_client	dim_client
pays	stg_pays	dim_client (la région)
region	stg_region	dim_client (la région)
produit	stg_produit	dim_produit
commande	stg_commande	fct_ventes (la date, le client)
ligne	stg_ligne	fct_ventes (le grain : une ligne = une vente)

pays et region fusionnent dans dim_client (on aplatit la géographie sur le client), la dénormalisation de l'étoile, vue à l'étape 2. Et ligne, la table la plus fine, donne le grain du fait : une ligne de caisse = une ligne de fct_ventes.

0.4 Créer le projet dbt brulerie

Avant toute donnée, il faut un projet. On refait ici, en condensé, ce que le Labo 00 a fait pour Boréal : mêmes gestes, autres noms. Chaque projet de la formation vit dans le sien, on ne remélange jamais boreal, brulerie et aurore.

Dans Snowflake d'abord, sous SYSADMIN, l'entrepôt et la base :

```
USE ROLE SYSADMIN;
CREATE WAREHOUSE IF NOT EXISTS BRULERIE_WH WAREHOUSE_SIZE = XSMALL AUTO_SUSPEND = 60;
CREATE DATABASE IF NOT EXISTS BRULERIE;
CREATE SCHEMA IF NOT EXISTS BRULERIE.DEV;
```

Puis, dans votre dossier de travail (env conda formations actif) :

```
dbt init brulerie
```

dbt init pose les mêmes questions qu'au Cours 00. On répond : adaptateur snowflake, votre identifiant de compte, votre utilisateur, rôle SYSADMIN, entrepôt BRULERIE_WH, base BRULERIE, schéma DEV, 4 threads.

Piège — le terminal figé après la dernière réponse. Si dbt init ne rend jamais la main et que ~/.dbt/profiles.yml semble ne pas avoir bougé, coupez avec Ctrl+C : le fichier est écrit et le projet est créé. C'est le blocage décrit au Labo 00, étape 5.

Le profiles.yml accueille maintenant un deuxième projet. Il existe déjà, Boréal l'a fait créer au Cours 00, et dbt init brulerie ne le remplace pas : il ajoute un bloc brulerie: à côté du bloc boreal:. Un seul fichier, un bloc par projet, et c'est la ligne profile: du dbt_project.yml de chaque projet qui dit lequel des deux lire.

```
# ~/.dbt/profiles.yml (extrait)
boreal:                                     # posé au Cours 00, on n'y touche pas
  target: dev
  outputs:
    dev:
      # ... Les mêmes clés que ci-dessous, avec BOREAL_WH et BOREAL

brulerie:                                  # ajouté par `dbt init brulerie`
  target: dev
  outputs:
    dev:
      type: snowflake
      account: <votre_compte>
      user: <votre_utilisateur>
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
      role: SYSADMIN
      warehouse: BRULERIE_WH
      database: BRULERIE
      schema: DEV
      threads: 4
      insecure_mode: true                  # seulement si Le Cours 00 en a eu besoin
```

Deux réglages du Cours 00 ne suivent pas tout seuls dans le bloc neuf, et c'est là que ça coince en pratique.

Le mot de passe, d'abord. `dbt init` vient de le réécrire en clair : comme au Cours 00, on remplace la ligne `password: "{{ env_var( 'SNOWFLAKE_PASSWORD' ) }}"`, puis on définit la variable dans l'Anaconda Prompt (pas d'espace autour du `=`, pas de guillemets) :

```
set SNOWFLAKE_PASSWORD=votreMotDePasse
```

C'est le même compte Snowflake qu'au Cours 00, donc la même valeur qu'alors. Mais `set` ne vaut que pour la session en cours : dans un terminal fraîchement ouvert, la variable est vide et `dbt debug` échoue sur l'authentification. À refaire à chaque ouverture, ou une fois pour toutes avec `setx SNOWFLAKE_PASSWORD "votreMotDePasse"`, suivi d'une réouverture du terminal.

Le contournement OSCP, ensuite. Si vous avez dû ajouter `insecure_mode: true` au Cours 00 pour que `dbt debug` cesse de geler, ajoutez-le de nouveau ici. Le réglage vit dans la sortie `dev` d'un profil, pas au sommet du fichier : le bloc `brulerie` : tout neuf ne l'a pas hérité. Symptôme identique (l'exécution s'arrête après `Registered adapter: snowflake...` sans jamais afficher `Connection test`), correctif identique (Labo 00, étape 6).

Il reste à déclarer le seul paquet dont on aura besoin (à l'étape 4, pour les clés de substitution). On n'en installe pas d'autre : un package se prend quand le besoin est là, et `dbt_utils` est la seule exception qu'on déclare d'entrée de jeu (Théorie 04, section 3). Dans `packages.yml`, à la racine du projet :

```
packages:
- package: dbt-labs/dbt_utils
  version: [ ">=1.1.0", "<2.0.0" ]
```

Puis `dbt deps` pour l'installer, et `dbt debug` pour vérifier la connexion, il doit répondre « All checks passed! ». Le squelette est en place : `models/`, `seeds/`, `macros/`, `snapshots/` et `tests/` existent déjà, vides.

`dbt init` a aussi écrit le `dbt_project.yml`, avec son en-tête : le nom du projet, le profil de connexion et l'emplacement de chaque sorte de fichier. Vous n'y toucherez pas. Tout ce que les étapes suivantes ajoutent (le bloc `seeds` : à l'étape 1, le bloc `models` : aux étapes 3 et 4) vient se greffer en dessous. Les blocs YAML de cette synthèse sont donc des extraits ; le fichier complet est en annexe.

`dbt init` ou `dbt seed` ? Deux verbes, deux moments, et on les confond d'autant plus qu'ils se suivent. `dbt init <nom>` crée le projet : le dossier, le `dbt_project.yml`, la connexion. On le lance une fois dans la vie d'un projet, jamais plus. `dbt seed` charge les CSV rangés dans `seeds/` : c'est une commande de tous les jours, qu'on relance chaque fois qu'un CSV change. « Seed » n'est pas un mot d'init : `dbt init seed` n'existe pas, et `dbt init brulerie` ne charge aucune donnée.

Une dernière pièce, recopiée telle quelle du Cours 03 : la macro `generate_schema_name`. Sans elle, `dbt` préfixerait tous les schémas par celui du profil et vous obtiendriez `DEV_STAGING` au lieu de `STAGING`. Créez `macros/generate_schema_name.sql` avec le contenu du Labo 03, étape 3 : le fichier est identique d'un projet à l'autre pour les trois projets de la formation, qui font tous le même choix. Ce n'est pas un fichier universel qu'on recopierait dans n'importe quel projet `dbt` (Labo 03, étape 3, pour la nuance).

Étape 1 — Le brut figé, chargé par `dbt seed` *(démarche du Cours 01)*

Chez Boréal, faire entrer le brut demandait un format de fichier, un stage et un `COPY INTO` (le brut arrive d'un système externe). Ce n'est pas la seule porte d'entrée possible : la Théorie 01, section 5, en compare six, toutes légitimes selon le contexte. La Brûlerie prend la plus simple, `dbt seed` : elle fige son brut dans le projet `dbt` lui-même, sous forme de petits CSV, et laisse `dbt` les charger.

Le choix se défend ici pour deux raisons, et deux seulement : les données sont minuscules, et elles ne changent jamais. Passé quelques milliers de lignes, ou dès qu'une source bouge, on revient au `COPY INTO` de Boréal (un dépôt Git n'est pas un entrepôt de données). Ce qui suit, en revanche, est rigoureusement identique d'un projet à l'autre : la porte d'entrée change, `STAGING` et `MARTS` ne bougent pas.

1.1 Les seeds : le brut versionné

Un seed, c'est un fichier CSV rangé dans `seeds/`, que `dbt` charge dans l'entrepôt avec la commande `dbt seed`. Nos six tables de caisse deviennent six fichiers, aux noms et aux valeurs bruts, colonnes abrégées, code `seg`, dates en texte, exactement comme la caisse les exporte :

```
brulerie/
└─ seeds/
    ├── region.csv      reg_id,reg_nom
    ├── pays.csv        pays_id,pays_nom,reg_id
    ├── client.csv      cli_id,cli_nom,pays_id,seg
    ├── produit.csv     prod_id,prod_nom,cat
    ├── commande.csv    cmd_id,cmd_date,cli_id
    └── ligne.csv       cmd_id,lig_no,prod_id,qte,prix,rem
```

On range les seeds dans le schéma `RAW`, et on garde en texte ce qui devra être typé plus tard, c'est un vrai brut, ce sera à `STAGING` de le mettre en forme. On le déclare une fois dans `dbt_project.yml`, en ajoutant un bloc `seeds` :

```
# dbt_project.yml (extrait) – on ajoute le bloc seeds:
seeds:
  brulerie:                # Le nom du projet, comme dans l'en-tête
    +schema: raw           # Les seeds atterrissent dans BRULERIE.RAW
    commande:              # un CSV, nommé sans son extension
      +column_types:
        cmd_id: string
        cmd_date: string
        cli_id: string
  ligne:
    +column_types:
      qte: string
      prix: string
      rem: string
```

Les quatre niveaux se lisent comme partout ailleurs dans ce fichier (Théorie 03, section 2) : le type de ressource (seeds:), le projet (brulerie:), la ressource visée (commande: , le nom du CSV sans son extension), puis les réglages en +. Un réglage posé sur le projet vaut pour tous les seeds, c’est le cas de +schema: raw ; un réglage posé sur un CSV ne vaut que pour lui.

Pourquoi forcer le texte. Par défaut, dbt seed devine le type de chaque colonne, il lirait 2024-01-15 comme une vraie date et 3.00 comme un nombre. On l’en empêche là où ça compte, avec +column_types : les dates et les montants restent du texte, comme dans un vrai export de caisse. Ainsi STAGING a un vrai travail de typage (texte → date, texte → nombre), au lieu d’hériter d’un brut déjà propre. Les identifiants, eux, on laisse dbt les deviner : qu’ils arrivent en nombre ou en texte ne change rien à la suite.

Puis, depuis le projet (env conda formations actif) :

```
dbt seed

dbt lit les six CSV et crée six tables dans BRULERIE.RAW. On vérifie :

SHOW TABLES IN SCHEMA BRULERIE.RAW;      -- Les 6 tables du brut
SELECT * FROM BRULERIE.RAW.CLIENT;          -- seg = P / PRO (à décoder plus tard)
SELECT * FROM BRULERIE.RAW.COMMANDE;        -- cmd_date = texte "2024-01-15"
```

1.2 Seed, source ou modèle : bien nommer ses entrées

Trois sortes de tables entrent en jeu dans un projet dbt, et on ne les référence pas de la même façon. La distinction est nette une fois posée :

Ce que c’est	Chargé par...	On l’appelle avec...
Un seed (brut figé dans le repo)	dbt seed (dbt le gère)	ref('client')
Une source (brut chargé hors dbt)	COPY INTO, un ETL... (dbt ne le gère pas)	source('brut', 'client')
Un modèle (que dbt calcule)	dbt run	ref('stg_client')

À retenir — le brut de La Brûlerie est un seed, donc un ref(). C’est la différence concrète avec Boréal, dont le brut arrive par COPY INTO (un système externe) et se déclare donc en source(). dbt ayant lui-même créé les tables de seed, il les traite comme ses propres objets : on les référence avec ref('client'), pas source(...). Même pipeline, autre porte d’entrée.

Une conséquence surprend en comparant les deux projets : La Brûlerie n’a aucun _sources.yml, là où Boréal en a un. Ce n’est pas un oubli. Un _sources.yml sert à décrire des tables que dbt n’a pas créées, pour pouvoir les appeler avec source() ; ici il n’y en a aucune, puisque dbt seed a tout chargé. Les deux projets déclarent la même chose (d’où part le pipeline), mais chacun dans le fichier qui correspond à sa porte d’entrée : seeds: dans dbt_project.yml pour La Brûlerie, _sources.yml pour Boréal.

Étape 2 — L'étoile, sur papier (démarche du Cours 02)

Avant d'écrire une ligne de dbt, on conçoit l'étoile. C'est la seule couche qui se décide. On la prend avec la méthode Kimball, en quatre décisions, dans l'ordre.

2.1 Pourquoi le brut est pénible à analyser — OLTP contre OLAP

La forme de la caisse est faite pour écrire, pas pour analyser. Posez une question d'analyste (« *quelles sont les ventes nettes par région ?* ») et deux obstacles surgissent :

1. Le contexte est éparpillé. Pour relier une vente à « Amérique » ou « Europe », il faut remonter une chaîne de tables : `ligne` → `commande` → `client` → `pays` → `region`. Cinq tables à joindre, pour une seule question.
2. La mesure n'existe pas toute faite. « Vente nette » = quantité × prix × (1 – remise). Nulle part elle n'est stockée : il faut la recalculer à chaque requête.

Un système d'analyse (OLAP) veut l'inverse d'un système d'application (OLTP) : il lit et additionne d'énormes quantités de lignes, et sa priorité est la lisibilité. Pour ça, on accepte de répéter l'information (« Europe » écrit sur chaque vente européenne) si ça supprime des jointures. La modélisation dimensionnelle réorganise ces mêmes données, sans rien perdre.

2.2 Les 4 décisions de Kimball

Décision 1 — Le processus métier : la vente. On modélise une activité concrète à la fois. Ici, la vente, celle dont on veut suivre les chiffres.

Décision 2 — Le grain : une ligne = un produit sur une commande. Le grain, c'est ce que représentera une seule ligne du fait. On le prend le plus fin possible : la ligne de commande. La commande 100 (Alice), qui a deux produits, donnera deux lignes de fait. Pourquoi si fin ? Parce qu'on agrège toujours vers plus gros (par commande, par jour, par mois), mais qu'on ne redescend jamais dans un détail qu'on n'a pas gardé.

Décision 3 — Les dimensions : le contexte (client, produit, date). Une dimension porte le contexte descriptif, large (beaucoup de colonnes) mais courte (peu de lignes). Deux mécanismes reviennent dans chacune :

- La clé de substitution — un identifiant que l'entrepôt invente. Chaque dimension reçoit une colonne `..._key` absente de la source. La clé métier (`client_id = 42`) vient du brut, on la garde ; la clé de substitution (`client_key`) est fabriquée, et c'est elle que le fait référencera. (Son utilité éclate à la section SCD, plus bas.)
- La dénormalisation — aplatir plusieurs tables en une. Le brut éclate le contexte ; la dimension le rassemble sur une ligne. On suit les liens de la source (`client` → `pays` → `region`) et on s'arrête au bon endroit : ce qui décrit → dimension ; ce qui se produit → fait (une commande est un événement, elle ira au fait).

`dim_client` — on aplatit `client` + `pays` + `region` :

<code>client_key</code>	<code>client_id</code>	<code>nom</code>	<code>segment</code>	<code>pays</code>	<code>region</code>
1	42	Alice	Particulier	Canada	Amérique

client_key	client_id	nom	segment	pays	region
2	57	Bruno	Pro	France	Europe
3	88	Carla	Particulier	France	Europe

dim_produit — pas de hiérarchie à remonter, on ajoute juste la clé :

produit_key	produit_id	nom	categorie
1	5	Café	Boisson
2	8	Thé	Boisson
3	9	Tasse	Accessoire

dim_date — la dimension qu'on fabrique, car la date ne vient d'aucune table. Une ligne par jour, avec des attributs pré-calculés ; sa clé de substitution est l'entier AAAAMMJJ, lisible et triable :

date_key	date	annee	trimestre	mois	nom_mois
20240115	2024-01-15	2024	T1	1	janvier
20240302	2024-03-02	2024	T1	3	mars

Décision 4 — Les faits : fct_ventes. On traduit chaque ligne du brut : on remplace chaque identifiant par la clé de substitution de sa dimension, et on calcule les mesures. Un fait ne contient que ça, des clés et des nombres, jamais de texte :

date_key	client_key	produit_key	quantite	montant_brut	montant_remise	montant_net	taux_remise
20240115	1	1	10	30,00	3,00	27,00	0,10
20240115	1	3	2	10,00	0,00	10,00	0,00
20240115	2	2	5	20,00	0,00	20,00	0,00
20240302	3	1	8	24,00	1,20	22,80	0,05
20240302	3	2	3	12,00	0,00	12,00	0,00

Première ligne : le 15 janvier (date_key 20240115), Alice (client_key 1) a acheté du Café (produit_key 1) ; montant_brut = $10 \times 3,00 = 30,00$; montant_remise = $30,00 \times 0,10 = 3,00$; montant_net = 27,00. L'étoile est complète : fct_ventes au centre, entouré de dim_date, dim_client, dim_produit.

2.3 La récompense : une question, une seule jointure

Sur le brut, « ventes nettes par région » demandait 5 jointures + un recalcul. Sur l'étoile, il suffit de coller le fait à une dimension :

```
SELECT c.region,
       SUM(f.montant_net) AS ventes_nettes
FROM   fct_ventes f
JOIN   dim_client c USING (client_key)  -- une seule jointure
GROUP BY c.region;
```

Le résultat, qu'on recompte à la main :

region	ventes_nettes	(le détail)
Amérique	37,00	Alice : 27,00 + 10,00
Europe	54,80	Bruno 20,00 + Carla (22,80 + 12,00)

2.4 Ce que le petit modèle laisse voir « en vrai »

L'additivité — toutes les mesures ne se somment pas. On additionne les montants sans problème (27,00 + 10,00 + 20,00 + 22,80 + 12,00 = 91,80 \$). Mais sommer taux_remise (0,10 + 0,05 = 0,15 = 15 %) est absurde : un taux n'est pas additif. La vraie remise moyenne = somme des remises 4,20 ÷ somme du brut 96,00 = 4,4 %. La règle Kimball : on stocke les composants additifs (montant_brut, montant_remise) et on recalcule les ratios à la lecture.

Type de mesure	Sommable sur...	Dans notre fait
Additive	toutes les dimensions	quantite, montant_brut, montant_remise, montant_net
Semi-additive	oui, sauf le temps	(aucune ici — l'exemple classique : un niveau de stock)
Non-additive	jamais directement	taux_remise (un ratio)

Étoile ou flocon. Notre dim_client est aplatie (pays et région dedans) : c'est l'étoile, une dimension à un saut du fait. Le flocon re-normaliserait la géographie en sous-tables (dim_client → dim_pays → dim_region), au prix de plus de jointures. Kimball tranche pour l'étoile ; et sur Snowflake, le stockage compressé en colonnes rend la redondance quasi gratuite.

Historiser un changement : les SCD. Le 1^{er} juin 2024, Carla passe de Particulier à Pro. En SCD type 1, on écrase (l'histoire est perdue). En SCD type 2, on historise, et c'est ici que la clé de substitution paie :

client_key	client_id	nom	segment	pays	region	valide_de	valide_a	est_courant
3	88	Carla	Particulier	France	Europe	2024-01-01	2024-05-31	faux

client_key	client_id	nom	segment	pays	region	valide_de	valide_a	est_courant
4	88	Carla	Pro	France	Europe	2024-06-01	9999-12-31	vrai

Même clé métier (88), nouvelle clé de substitution (4). La vente de Carla en mars pointe vers `client_key` 3 → elle reste « Particulier » : la vérité d'époque est préservée. Voilà pourquoi on a inventé une clé de substitution au lieu de réutiliser `client_id` : une même personne a maintenant deux versions, et il faut une clé différente par version pour que le fait pointe vers la bonne.

Dimensions conformes. Si La Brûlerie suivait aussi ses retours, un second fait `fct_retours` réutiliserait exactement les mêmes `dim_date`, `dim_client`, `dim_produit` : des dimensions conformes. On croiserait alors ventes et retours par produit sans rien réconcilier, parce que les deux faits parleraient du même produit et de la même date.

Étape 3 — STAGING en dbt : le brut, rendu lisible (*démarche du Cours 03*)

À l'étape 2, on a raisonné sur un brut déjà propre (`client_id`, `date_commande...`), pour se concentrer sur l'étoile. Le vrai brut, lui, est cryptique (`cli_id`, `seg`, `dates` en texte). STAGING est l'endroit où naissent, une fois pour toutes, les noms propres et les bons types.

3.1 Ce que dbt fait

dbt ne stocke aucune donnée : il génère du SQL et l'envoie à Snowflake. L'unité de travail est le modèle, un fichier `.sql` contenant un seul `SELECT`. Vous écrivez le `SELECT` ; dbt l'enveloppe dans un `CREATE VIEW` (ou `CREATE TABLE`) et l'exécute. Le SQL est enrichi de Jinja : les `{{ ... }}` sont évalués par dbt avant l'envoi, c'est ce qui permet `ref()` et `source()`.

3.2 Régler la couche d'un coup

Comme au Labo 03, on règle la couche avant d'écrire le premier modèle : toute la couche staging est en materialization `view` (léger, toujours frais) et atterrit dans le schéma `STAGING`. Deux lignes dans `dbt_project.yml` suffisent, et aucun modèle n'aura à les répéter.

```
# dbt_project.yml (extrait) – on ajoute le bloc models:
models:
  brulerie:
    staging:
      +materialized: view      # toute la couche staging = des vues
      +schema: staging        # → schéma STAGING
```

C'est la macro `generate_schema_name` posée à l'étape 0.4 qui garantit `STAGING` tout court, et non `DEV_STAGING`.

3.3 Un modèle de staging, décortiqué

stg_client renomme et décode, rien de plus (pas de jointure, pas de logique métier, du 1 pour 1). Il lit le seed client avec ref() (rappel de l'étape 1 : le brut de La Brûlerie est un seed) :

```
-- models/staging/stg_client.sql
with source as (
  select * from {{ ref('client') }}           -- un seed → ref(), pas source()
)
select
  cli_id          as client_id,
  cli_nom         as nom_client,
  pays_id        as pays_id,
  case seg when 'P' then 'Particulier'
           when 'PRO' then 'Pro' end as segment
from source
```

dbt évalue la Jinja (ref('client') → BRULERIE.RAW.CLIENT) et enveloppe le tout dans un CREATE VIEW BRULERIE.STAGING.STG_CLIENT AS Le résultat, quand on lit la vue, enfin des noms propres et un segment décodé :

client_id	nom_client	pays_id	segment
42	Alice	1	Particulier
57	Bruno	2	Pro
88	Carla	2	Particulier

C'est une vue : elle ne stocke rien ; à chaque lecture, Snowflake rejoue le select. Donc si le brut change, stg_client est toujours à jour, gratuitement, c'est tout l'intérêt du +materialized: view posé à la section précédente.

3.4 Les six modèles, d'un coup d'œil

Un .sql par table du brut, toujours le même moule (with source as (...) select ... rename). La convention de nom est stg_<source> :

Modèle	lit le seed	fait surtout...
stg_region	region	renommer reg_id → region_id, reg_nom → nom_region
stg_pays	pays	renommer pays_id, pays_nom → nom_pays, reg_id → region_id
stg_client	client	renommer + décoder seg → segment (P/PRO)

Modèle	lit le seed	fait surtout...
stg_produit	produit	renommer prod_id → produit_id, prod_nom → nom_produit, cat → categorie
stg_commande	commande	renommer + typer la date (to_date(cmd_date))
stg_ligne	ligne	renommer + typer les nombres (qte → quantite...) — le futur grain du fait

Les deux qui typent (au-delà du simple renommage) :

```
-- models/staging/stg_commande.sql → La date texte redevient une vraie date
with source as ( select * from {{ ref('commande') }} )
select
  cmd_id                as commande_id,
  to_date(cmd_date, 'YYYY-MM-DD') as date_commande,  -- texte "2024-01-15" → DATE
  cli_id                as client_id
from source

-- models/staging/stg_ligne.sql → Les nombres texte redeviennent des nombres
with source as ( select * from {{ ref('ligne') }} )
select
  cmd_id                as commande_id,
  lig_no                as ligne_num,
  prod_id               as produit_id,
  to_number(qte)        as quantite,
  to_decimal(prix, 10, 2) as prix_unitaire,
  to_decimal(rem, 4, 2)  as taux_remise
from source
```

Chaque stg_* reste 1 pour 1 avec sa table : autant de lignes en sortie qu’en entrée, aucune jointure. C’est une façade stable sur le brut, le jour où la caisse renomme cli_id, on corrige un modèle, et tout l’aval n’y voit que du feu.

3.5 ref() et le graphe (le DAG)

En reliant tous les ref(), dbt bâtit le DAG (graphe des dépendances) et déduit l’ordre d’exécution, vous ne le dictez jamais. Ici, le seed nourrit le staging, qui nourrira les marts :

RAW (seeds)	→ ref() →	STAGING	→ ref() →	MARTS (étape 4)
client	→	stg_client	→	dim_client

RAW (seeds)	→ ref() →	STAGING	→ ref() →	MARTS (étape 4)
pays	→	stg_pays	→	dim_client
region	→	stg_region	→	dim_client
ligne	→	stg_ligne	→	fct_ventes
commande	→	stg_commande	→	fct_ventes

On construit la couche : `dbt run --select staging`. Six modèles au vert, matérialisés en view dans `BRULERIE.STAGING`.

Étape 4 — MARTS en dbt : l'étoile, assemblée (démarche du Cours 04)

Le staging était en vues (léger, toujours frais). Les marts, lus souvent (par Power BI, par les analystes), se figent en table. Chaque mart lit le staging avec `ref()`, jamais le brut :

```
# dbt_project.yml (extrait) – on complète le bloc models: de l'étape 3.2
models:
  brulerie:
    staging:
      +materialized: view
      +schema: staging
  marts:
    +materialized: table      # Les marts sont figés en tables
    +schema: marts           # → schéma MARTS
```

On n'a rien réécrit : on a ajouté un dossier, `marts:`, à côté du `staging:` déjà réglé. Le fichier grossit couche par couche, jamais d'un bloc.

4.1 La clé de substitution par hachage — l'idée maîtresse

À l'étape 2, on forgeait la clé de substitution avec un `ROW_NUMBER()` (un entier). En dbt, on préfère un hachage de la clé métier, via `{{ dbt_utils.generate_surrogate_key(['client_id']) }}` (du package `dbt_utils`, installé par `dbt deps`). Pour `client_id = 42`, elle produit toujours le même hachage, par exemple `a1d0c6e8...` (*md5 abrégé, illustratif*).

La raison : un hachage est recalculable partout à partir de la clé métier. Conséquence décisive (section 4.4) : le fait peut recalculer `client_key = hash(client_id)` de son côté, sans joindre `dim_client`, les deux hachent le même `client_id`, donc obtiennent la même valeur.

4.2 dim_produit — la plus simple

Elle lit une vue de staging et ajoute sa clé :

```
-- models/marts/dim_produit.sql
with produits as ( select * from {{ ref('stg_produit') }} )
select
    {{ dbt_utils.generate_surrogate_key(['produit_id']) }} as produit_key,
    *
from produits
```

4.3 dim_client — on aplatit trois vues de staging

ref() relie plusieurs modèles : on joint stg_client + stg_pays + stg_region pour aplatir la géographie (comme à l'étape 2, mais à partir du staging), puis on hache la clé :

```
-- models/marts/dim_client.sql
with clients as ( select * from {{ ref('stg_client') }} ),
    pays      as ( select * from {{ ref('stg_pays') }} ),
    regions   as ( select * from {{ ref('stg_region') }} ),
joint as (
    select clients.client_id, clients.nom_client, clients.segment,
           pays.nom_pays as pays, regions.nom_region as region
    from clients
    join pays      on pays.pays_id      = clients.pays_id
    join regions   on regions.region_id = pays.region_id
)
select
    {{ dbt_utils.generate_surrogate_key(['client_id']) }} as client_key,
    *
from joint
```

dim_date se fabrique avec dbt_utils.date_spine (la liste des jours entre deux bornes), qu'on enveloppe pour en tirer la clé AAAAMMJJ et les attributs, plus de boucle à écrire à la main.

4.4 fct_ventes — le fait, sans jointure aux dimensions

Le fait lit stg_ligne (le grain) et stg_commande (pour la date et le client), puis recalcule ses clés par le même hachage. C'est ici que le choix du hachage paie :

```
-- models/marts/fct_ventes.sql
with lignes      as ( select * from {{ ref('stg_ligne') }} ),
    commandes    as ( select * from {{ ref('stg_commande') }} ),
joint as (
    select c.date_commande, c.client_id, l.produit_id,
           l.commande_id, l.ligne_num,           -- Le grain, porté par le fait
           l.quantite, l.prix_unitaire, l.taux_remise
    from lignes l
```

```

    join commandes c on c.commande_id = l.commande_id
)
select
    to_number(to_char(date_commande, 'YYYYMMDD')) as date_key,
    {{ dbt_utils.generate_surrogate_key(['client_id']) }} as client_key,
    {{ dbt_utils.generate_surrogate_key(['produit_id']) }} as produit_key,
    commande_id,
    ligne_num,
    quantite,
    quantite * prix_unitaire as montant_brut,
    quantite * prix_unitaire * taux_remise as montant_remise,
    quantite * prix_unitaire * (1 - taux_remise) as montant_net,
    taux_remise
from joint

```

Le fait a écrit client_key = a1d0c6e8 pour Alice sans jamais lire dim_client, il a juste haché client_id = 42. Et dim_client contient le même a1d0c6e8, parce qu'elle a haché le même 42. Les deux se rejoignent tout seuls. Voilà pourquoi on préfère le hachage à un ROW_NUMBER : pas de recherche dans la dimension.

À retenir — La Brûlerie reste au socle. Son fait a le prix dans stg_ligne : rien à assembler, il va directement stg → fct, sans couche INTERMEDIATE. Chez Boréal, le coût vit dans une autre table et doit être joint pour calculer la marge : cet assemblage est factorisé dans une couche int_ventes_enrichies. C'est la dose de plus de Boréal, La Brûlerie, elle, montre le socle nu.

Étape 5 — La confiance : tests, historique, documentation (démarche du Cours 04)

C'est ce qui sépare un livrable d'un SQL qui tourne.

5.1 Tester

dbt teste de deux façons. Les tests génériques se déclarent dans un .yaml, à côté des descriptions :

```

# models/marts/_marts.yaml (extrait)
models:
  - name: dim_client
    description: "Un client par ligne, géographie aplatie."
    columns:
      - name: client_key
        tests: [unique, not_null] # clé de dimension : unique et non nulle
  - name: fct_ventes
    description: "Une ligne par produit vendu (grain : commande_id + ligne_num)."

```

```

tests:
  - dbt_utils.unique_combination_of_columns: # Le grain composite, prouvé
    arguments:
      combination_of_columns: [commande_id, ligne_num]
columns:
  - name: client_key
    tests:
      - not_null
      - relationships: # Le gardien de l'étoile
        arguments:
          to: ref('dim_client')
          field: client_key

```

Ce qu'ils garantissent : unique/not_null interdisent une clé de dimension en double ou vide ; relationships prouve que chaque client_key du fait existe bien dans dim_client (l'intégrité référentielle, aucune vente vers un client fantôme) ; et unique_combination_of_columns prouve le grain, aucune paire (commande_id, ligne_num) en double. Pour une règle sur mesure, un test singulier, un SELECT qui renvoie les lignes fautives (zéro ligne = réussi) :

```

-- tests/assert_montant_net_non_negatif.sql
select * from {{ ref('fct_ventes') }} where montant_net < 0

```

5.2 Historiser Carla : un snapshot (SCD2 automatique)

À l'étape 2, on a historisé un changement de segment à la main (deux versions, colonnes de validité). dbt le fait tout seul avec un snapshot : à chaque exécution, il compare l'état courant à l'état stocké et, s'il détecte un changement, ferme l'ancienne version et insère la nouvelle :

```

-- snapshots/scd_client.sql
{% snapshot scd_client %}
{{ config(target_schema='snapshots', unique_key='client_id',
      strategy='check', check_cols=['segment']) }}
select client_id, nom_client, segment from {{ ref('stg_client') }}
{% endsnapshot %}

```

Une première photo (dbt snapshot) ; puis Carla passe de Particulier à Pro ; un dbt snapshot de plus, et BRULERIE.SNAPSHOTS.SCD_CLIENT montre deux versions, dbt_valid_from/dbt_valid_to remplies par dbt. C'est le SCD type 2 de l'étape 2, sans rien écrire à la main.

Piège — un snapshot accumule. On le lance avec dbt snapshot, jamais dbt run, et on ne le reconstruit jamais de zéro (--full-refresh sur un snapshot détruirait l'historique).

5.3 Documenter et tout lancer

- Documentation : chaque modèle a une description dans le .yaml. dbt docs generate puis dbt docs serve produisent un site avec le graphe de lignage cliquable, du brut aux marts. Une doc qui ne se périmé pas (générée du code).

- `dbt build` : la commande de tous les jours. Elle parcourt le DAG et, pour chaque modèle, le construit puis le teste immédiatement (et joue les snapshots). Un test rouge arrête la branche, une donnée fausse ne se propage pas en aval.

Étape 6 — Le faire tourner (*démarche du Cours 05*)

L'étoile est montée une fois. Voici comment elle tourne en régime permanent, en deux gestes.

Un lot mensuel, en incrémental. Le 1^{er} avril, la caisse exporte deux commandes de plus. Plutôt que de tout refaire, on passe `fct_ventes` en `incremental` : il ne recalcule que les lignes plus récentes que ce qu'il contient déjà, en `MERGE` sur le grain (`unique_key = [commande_id, ligne_num]`, `where date_key > (select max(date_key) from {{ this }}`) sous `{% if is_incremental() %}`). Rejouer sans lot neuf ne fusionne rien : le pipeline est planifiable en confiance. (*Avec des seeds, un « lot » se simule en ajoutant des lignes aux CSV puis en relançant `dbt seed` ; chez Boréal, c'est un `COPY INTO` de plus.*)

Deux environnements, dev et prod. La table qu'Alice consulte ne doit pas être celle où l'on bricole. On sépare dev et prod par la cible (`dbt build` → `DEV_MARTS` ; `dbt build --target prod` → `MARTS`), via une macro `generate_schema_name` consciente de l'environnement. Un seul projet, deux cibles. Orchestrer, enfin, c'est planifier `dbt build --target prod` (planificateur système, `dbt Cloud`, `Airflow`...) : `dbt` fournit déjà le quoi et l'ordre (le DAG), il ne manque que le quand.

Récapitulatif — tout l'arc, en un tableau

Démarche (cours d'origine)	Où, dans cette synthèse
Architecture en couches, socle RAW → STAGING → MARTS (00)	Étape 0.3
Créer le projet dbt et sa connexion (<code>dbt init</code> , profil, <code>dbt deps</code>) (00)	Étape 0.4
Ingestion — le brut figé chargé par <code>dbt seed</code> (01)	Étape 1
Seed vs source vs modèle (ref vs source) (01, 03)	Étape 1.2
OLTP vs OLAP, les 4 décisions de Kimball (02)	Étape 2.1-2.2
Grain fin, clé de substitution, dénormalisation (02)	Étape 2.2
Additivité, étoile/flocon, SCD, dimensions conformes (02)	Étape 2.4
<code>dbt</code> : modèle, Jinja, materialization view (03)	Étape 3
<code>dbt_project.yml</code> : en-tête, blocs <code>seeds</code> : et <code>models</code> : (03)	Étapes 1, 3.2, 4 et annexe

Démarche (cours d'origine)	Où, dans cette synthèse
STAGING : 6 vues stg_* (renommer, décoder, typer) (03)	Étape 3.3-3.4
ref() et le DAG (03)	Étape 3.5
MARTS : dimensions + fait en table (04)	Étape 4
Clé par hachage → le fait ne joint pas les dimensions (04)	Étape 4.1, 4.4
Tests génériques + grain composite + test singulier (04)	Étape 5.1
Snapshot = SCD2 automatique (04)	Étape 5.2
Documentation, lignage, dbt build (04)	Étape 5.3
Incrémental, dev/prod, orchestration (05)	Étape 6

En un mot : le même pipeline que Boréal, réduit à l'os, une source, un processus, cinq lignes de vente. Rien ne s'y cache : on recompte chaque total de tête. Une fois cet arc lisible d'un coup d'œil, tout ce que la formation a posé séparément tient ensemble dans un seul document.

Annexe — l'arborescence du projet et les commandes

Le projet dbt de La Brûlerie, brulerie (base BRULERIE), tient dans une poignée de dossiers :

```
brulerie/
├─ dbt_project.yml          # config : seeds → RAW, staging → vues, marts → tables
├─ packages.yml            # dbt_utils
├─ seeds/                  # le brut figé (6 CSV), chargé par `dbt seed`
│  ├─ region.csv   pays.csv   client.csv
│  └─ produit.csv  commande.csv ligne.csv
├─ macros/
│  └─ generate_schema_name.sql # schémas propres (STAGING, MARTS) – copie du Cours 03
├─ models/
│  ├─ staging/          # 6 vues stg_* ; pas de _sources.yml : le brut est un seed
│  │  ├─ stg_region.sql ... stg_ligne.sql
│  │  └─ marts/
│  │     ├─ dim_client.sql  dim_produit.sql  dim_date.sql  fct_ventes.sql
│  │     └─ _marts.yml      # descriptions + tests
├─ snapshots/
│  └─ scd_client.sql
```

```
└─ tests/
  └─ assert_montant_net_non_negatif.sql
```

Et le `dbt_project.yml` en entier — les extraits des étapes 1, 3.2 et 4 mis bout à bout, sous l'en-tête qu'a écrit `dbt init`. C'est le seul fichier de config du projet ; il n'y en a pas d'autre à créer.

```
name: 'brulerie'
version: '1.0.0'
profile: 'brulerie'

model-paths: ["models"]
analysis-paths: ["analyses"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

clean-targets:
  - "target"
  - "dbt_packages"

# --- ce que vous ajoutez : l'étape 1 ---
seeds:
  brulerie:
    +schema: raw
    commande:
      +column_types:
        cmd_id: string
        cmd_date: string
        cli_id: string
    ligne:
      +column_types:
        qte: string
        prix: string
        rem: string

# --- ce que vous ajoutez : les étapes 3.2 et 4 ---
models:
  brulerie:
    staging:
      +materialized: view
      +schema: staging
  marts:
```

```
+materialized: table
+schema: marts
```

Comparé à celui de Boréal, ce fichier a un bloc seeds : que Boréal n'a pas, et pas de dossier intermediate : , La Brûlerie n'en a pas eu besoin. L'en-tête, lui, est le même à un nom près.

Pour tout rejouer dans votre compte Snowflake, depuis le dossier brulerie/ (env conda formations actif, SNOWFLAKE_PASSWORD posé) :

```
dbt deps                # installe dbt_utils
dbt seed                # charge le brut figé → BRULERIE.RAW
dbt run                 # bâtit staging (vues) puis marts (tables)
dbt snapshot            # première photo SCD2 de dim_client
dbt test                # joue tous les tests
# ou, en un geste :
dbt build               # seed + run + snapshot + test, dans l'ordre du DAG
```

La dernière requête de contrôle doit renvoyer Amérique 37,00 et Europe 54,80, exactement les chiffres calculés à la main. La boucle est bouclée.

Le corrigé complet est à côté, dans Solution/ : Projet_Brulerie.sql pour le SQL Snowflake et Solution/dbt/ pour le projet dbt entier, seeds compris. Allez-y après avoir cherché, ou pour comparer, une fois votre version debout.